

seL4 Self Test Suite

General

Intention of this page: collect useful observations about seL4_test

Executing a subset of the tests

The tests comprising seL4_test are not contained in one module. They can be found by searching for the the C-macro "DEFINE_TEST". Here is an exemplary use:

```
DEFINE_TEST(IPC0002, "Test SMP seL4_Call + seL4_ReplyRecv", test_call_replywait, true)
```

This macro allows to provide a bool indicating whether the test is active or not. In theory this parameter can be used to define a set of tests which should be executed by seL4_test. As there are currently more than hundred tests defined this approach is a bit cumbersome.

A simpler method is to select the tests to be executed using their name at the central test filtering in seL4_test (/apps/sel4test-driver/src/main.c function collate_tests). Below is an example which makes seL4_test execute only one test selected by its name:

```
+static bool takeTest(const char *testName)
+{
+    if (strcmp("IPC0002", testName) == 0)
+        return true;
+    return false;
+}
+
+static int collate_tests(testcase_t *tests_in, int n, testcase_t *tests_out[], int out_index,
+                        regex_t *reg, int* skipped_tests)
+{
+    @@ -299,7 +307,10 @@ static int collate_tests(testcase_t *tests_in, int n, testcase_t *tests_out[], i
+    /* make sure the string is null terminated */
+    tests_in[i].name[TEST_NAME_MAX - 1] = '\0';
+    if (regexexec(reg, tests_in[i].name, 0, NULL, 0) == 0) {
+    +#if 0
+        if (tests_in[i].enabled) {
+    +#endif
+        if (takeTest(tests_in[i].name)) {
+            tests_out[out_index] = &tests_in[i];
+            out_index++;
+        } else {
```

Architecture / implementation notes

- The build process of seL4_test actually creates two elf files:
 - sel4test-driver (startup code: sel4test/apps/sel4test-driver/src/main.c): contains the execution / control logic of the test suite
 - sel4test-tests (startup code: sel4test/apps/sel4test-tests/src/main.c): contains the code of all the tests
- sel4test-driver is the user image when booting seL4
- sel4test-driver:
 - starts a new process (= seL4 thread with separate address space) for every single test using the elf file of sel4test-tests; there seems to be the option to start tests in parallel
 - passes as argument the name of the test to be executed; this info is used by sel4test-tests to find the function that needs to be called for its invocation
 - passes as argument an seL4 endpoint, which:
 - sel4test-tests uses to "return" (send) the test verdict
 - serves sel4test-tests as a server facility (sel4test-driver acts as a kind of timer server for a started test)
 - is set up as fault endpoint for sel4test-tests; which means that every exception arising in sel4test-tests will be sent by seL4 to this endpoint; so sel4test-driver uses this endpoint not just to wait for a test result - it is also prepared to receive an exception as a result of a problem in the execution of a test
- TODO: tests can (and many do) create internal helper threads; document how the communication is implemented in these scenarios